InstantNGP vs 3D Gaussian Splatting on a Self-Captured Scene

1. Objective

Compare InstantNGP [1] and 3D Gaussian Splatting (3DGS) [2] on one self-captured static scene with respect to:

1. reconstruction quality on held-out test views (PSNR, SSIM, LPIPS, visual inspection),
2. rendering time,
3. model size,
4. training time.

Both methods use the same images, camera poses, train/test split, resolution, GPU and evaluation code.

2. Dataset

2.1 Capture

Item	Value
Scene	wall-mounted shelf with books, ornaments, plants, trophies (static, indoor)
Device / mode	phone camera, video
Video	1080 x 1920 (portrait), 30 fps, HEVC, 31.1 s
Camera motion	one sideways pass along the shelf at constant height and distance, without rotating the camera
Lighting	indoor room lighting

2.2 Frame selection

Step	Setting	Result
Extraction (ffmpeg)	6 fps, resized to 720 x 1280 (Lanczos)	186 candidate frames
Sharpness filter	variance of the Laplacian; sharpest frame kept in each group of 3 consecutive frames	62 frames
Visual inspection	blur, moving objects, photographer shadow	62 frames used

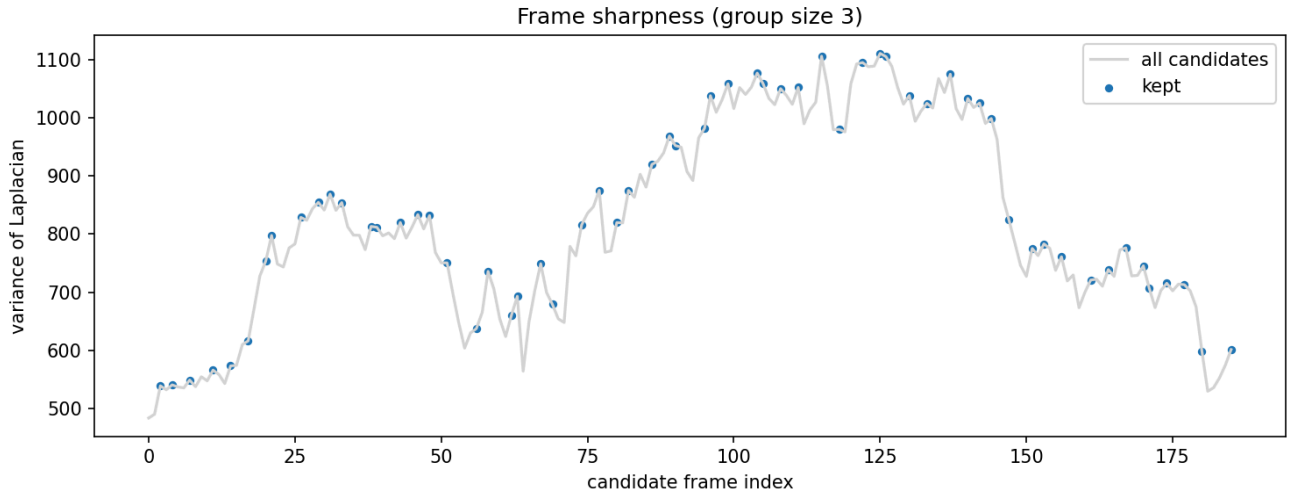


Figure 1. Sharpness (variance of the Laplacian) of all candidate frames; kept frames highlighted.

2.3 Camera poses (COLMAP)

COLMAP 4.2.0 (CPU) through the `convert.py` script of the 3DGS repository: SIFT features, exhaustive matching, incremental mapping with a single shared OPENCV camera, then undistortion to a PINHOLE model.

Quantity	Value
Registered images	62 / 62
Sparse 3D points	16,573
Mean reprojection error	0.40 px
Undistorted image size	701 x 1249
Intrinsics (PINHOLE)	fx = 1147.75, fy = 1161.89, cx = 350.5, cy = 624.5
Camera path: spread along its 3 principal axes (std)	3.70 / 0.23 / 0.05 (COLMAP units)
Deviation of viewing directions from their mean	mean 4.1°, max 8.5°
Path length / median scene depth	12.58 / 6.22 (COLMAP units)

The camera centres lie on an almost straight horizontal line with nearly parallel viewing directions, as required by the capture protocol.

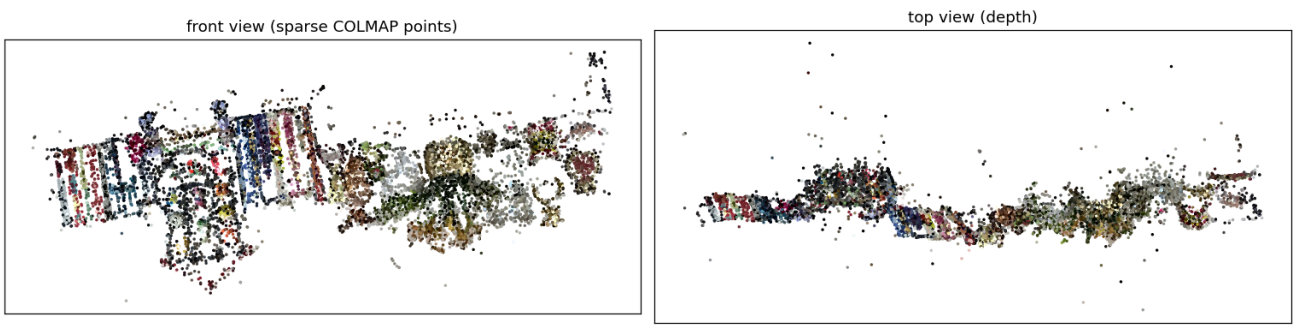


Figure 2. COLMAP sparse point cloud (front and top view).

2.4 Train / test split

Images are sorted by name and every 8th image (index mod 8 = 0) is held out for testing, which is the rule 3DGS uses with `--eval`.

Set	Images
Train	54
Test	8: 0000, 0008, 0016, 0024, 0032, 0040, 0048, 0056

Both methods train on the undistorted 701 x 1249 images and are evaluated against the same undistorted test images.

3. Methods and settings

3.1 3D Gaussian Splatting

Setting	Value
Code	graphdeco-inria/gaussian-splatting, commit `54c035f`
Initialisation	16,573 COLMAP points
Iterations	30,000 (official default)
Resolution	full (`-r 1`), 701 x 1249
Other hyper-parameters	repository defaults (SH degree 3, densification until 15k, opacity reset every 3k)
Checkpoints saved	1k, 3k, 7k, 15k, 30k

3.2 InstantNGP

Setting	Value
Code	NVlabs/instant-ngp, commit `abe236e`, default network `configs/nerf/base.json`
Encoding	multiresolution hash grid: 8 levels x 4 features, table size 2^19, base resolution 16
MLPs	density: 1 hidden layer x 64; colour: 2 hidden layers x 64
Steps	35,000 (default schedule) and continued to 100,000 to confirm convergence
`aabb_scale`	4
Checkpoints saved	1k, 3k, 7k, 15k, 35k, 100k
Rendering for evaluation	8 samples per pixel, black background (as in `run.py`)

Pose normalisation. `colmap2nerf.py` centres the scene at the point where all optical axes intersect. With a sideways capture the axes are almost parallel, and this point fell 3.4 units (normalised scale) behind the wall, leaving the shelf far from the centre of InstantNGP's volume. The poses were therefore recentred on the median of the COLMAP points and scaled so the farthest camera is 1 unit from the centre. This is a similarity transform; image content, intrinsics and relative poses are unchanged.

4. Evaluation protocol

Item	Protocol
Metrics	one script for both methods. PSNR per image (images in [0,1]); SSIM with an 11x11 Gaussian window, σ = 1.5 (same implementation as 3DGS); LPIPS v0.1 with VGG and AlexNet backbones, inputs in [-1,1]. Values are averaged over the 8 test views.
Metric check	the official 3DGS `metrics.py`, run on the same renders, gives PSNR 36.030 and SSIM 0.9789, identical to our values
Training time	wall-clock time of the full training process, including data loading and checkpoint saving
Rendering time	mean over 8 test views x 20 repeats after 10 warm-up renders, at 701 x 1249. 3DGS: CUDA-synchronised around `render()`. InstantNGP: `testbed.render()`, which returns the image to host memory.
Model size	file on disk: 3DGS `point_cloud.ply`; InstantNGP `.ingp` snapshot (zlib-compressed by default)
GPU memory	peak of `nvidia-smi` memory used, sampled once per second during training (whole GPU)
Hardware	NVIDIA GeForce RTX 4060 Laptop GPU (8 GB, compute capability 8.9), driver 591.44, WSL2 Ubuntu 22.04, gcc 11.4

Sizes are given in MB = 2^20 bytes.

5. Results

5.1 Summary

	InstantNGP 35k	InstantNGP 100k	3DGS 30k
PSNR (dB) ↑	32.92	33.05	**36.03**
SSIM ↑	0.9590	0.9598	**0.9789**
LPIPS-VGG ↓	0.1626	0.1618	**0.1469**
LPIPS-Alex ↓	0.0606	0.0595	**0.0376**
Training time (wall clock)	**5.0 min** (301 s)	13.9 min (834 s)	43.2 min (2594 s)
Render time, quality setting	155.8 ms (6.4 FPS, 8 spp)	155.8 ms (6.4 FPS, 8 spp)	**17.4 ms (57.6 FPS)**
Render time, 1 spp	20.3 ms (49.2 FPS)	20.3 ms (49.2 FPS)	–
Model file	**23.5 MB**	**23.4 MB**	271.8 MB
Trainable parameters	12.87 M	12.87 M	67.79 M (1,148,987 Gaussians x 59)
Peak GPU memory during training	–	1209 MiB	4305 MiB

Render times were measured on the 100k model; the network size, and therefore the cost per sample, is the same at 35k. All quality numbers for InstantNGP are from 8 spp renders, so the 8 spp time is the one that matches its reported quality.

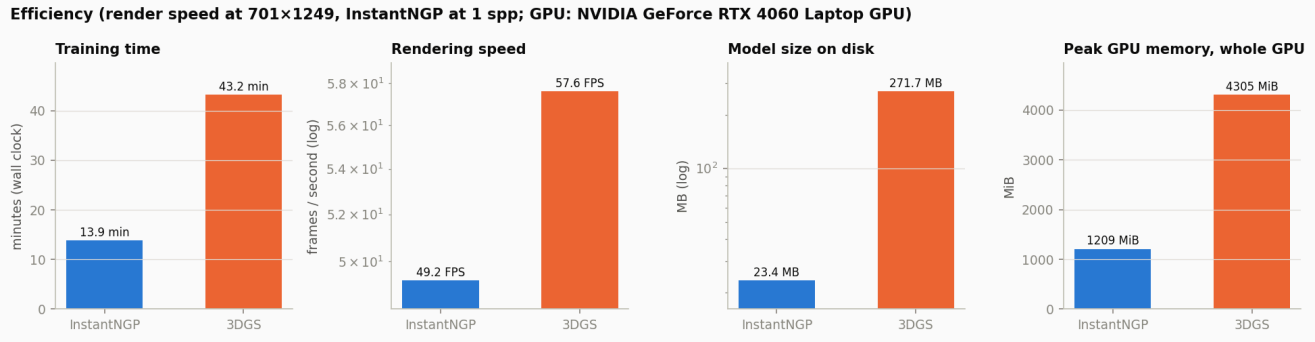


Figure 3. Training time, rendering speed (InstantNGP at 1 spp), model size and peak GPU memory.

5.2 Per-view quality

View	PSNR NGP	PSNR 3DGS	Δ PSNR	SSIM NGP	SSIM 3DGS	LPIPS-VGG NGP	LPIPS-VGG 3DGS
0000	33.15	36.03	+2.88	0.9557	0.9783	0.2036	0.1759
0008	33.73	37.31	+3.58	0.9653	0.9826	0.1593	0.1435
0016	33.26	36.53	+3.27	0.9624	0.9821	0.1624	0.1434
0024	29.21	31.15	+1.93	0.9343	0.9605	0.1823	0.1634
0032	34.08	37.55	+3.46	0.9678	0.9840	0.1482	0.1470
0040	32.74	35.52	+2.77	0.9622	0.9796	0.1404	0.1327
0048	33.38	36.11	+2.73	0.9642	0.9809	0.1475	0.1330
0056	34.84	38.05	+3.21	0.9669	0.9829	0.1506	0.1366
Mean	**33.05**	**36.03**	**+2.98**	**0.9598**	**0.9789**	**0.1618**	**0.1469**

NGP = InstantNGP at 100k steps; 3DGS at 30k iterations; Δ = 3DGS − NGP.

3DGS is better on all 8 views and all four metrics. The PSNR gap ranges from 1.93 to 3.58 dB.

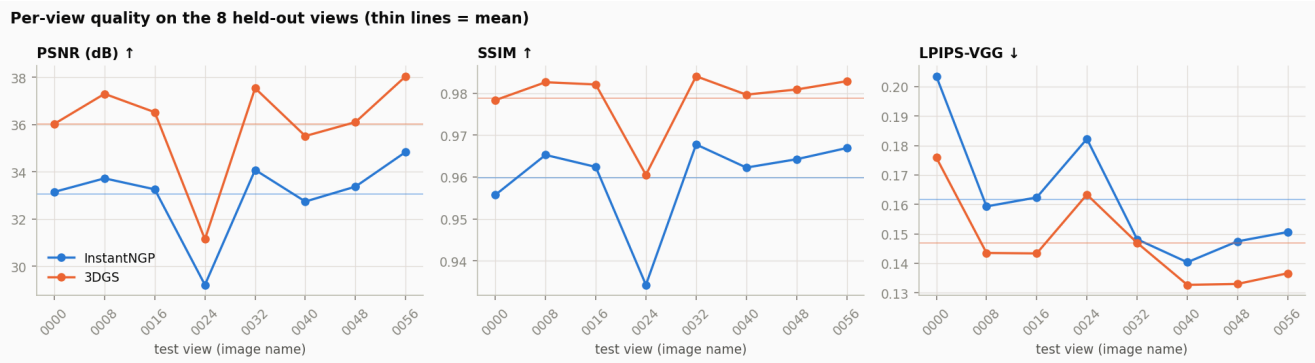


Figure 4. Per-view PSNR, SSIM and LPIPS-VGG (horizontal lines: means).

5.3 Convergence

3DGS iteration	1k	3k	7k	15k	30k
Wall time (min)	0.5	2.0	7.2	20.5	43.2
PSNR (dB)	27.57	32.48	35.08	35.95	36.03
SSIM	0.9151	0.9649	0.9765	0.9791	0.9789
LPIPS-VGG	0.2520	0.1732	0.1557	0.1498	0.1469
Gaussians	75,419	725,555	1,061,681	1,148,987	1,148,987

InstantNGP step	1k	3k	7k	15k	35k	100k
Wall time (min)	0.2	0.5	1.0	2.2	5.0	13.9
PSNR (dB)	26.59	29.28	30.74	31.73	32.92	33.05
SSIM	0.8850	0.9265	0.9427	0.9514	0.9590	0.9598
LPIPS-VGG	0.2685	0.2106	0.1864	0.1741	0.1626	0.1618

Both methods have converged: - **3DGS:** +0.08 dB PSNR and −0.0002 SSIM from 15k to 30k. - **InstantNGP:** +0.13 dB PSNR and +0.0008 SSIM from 35k to 100k, for 2.8x the training time.

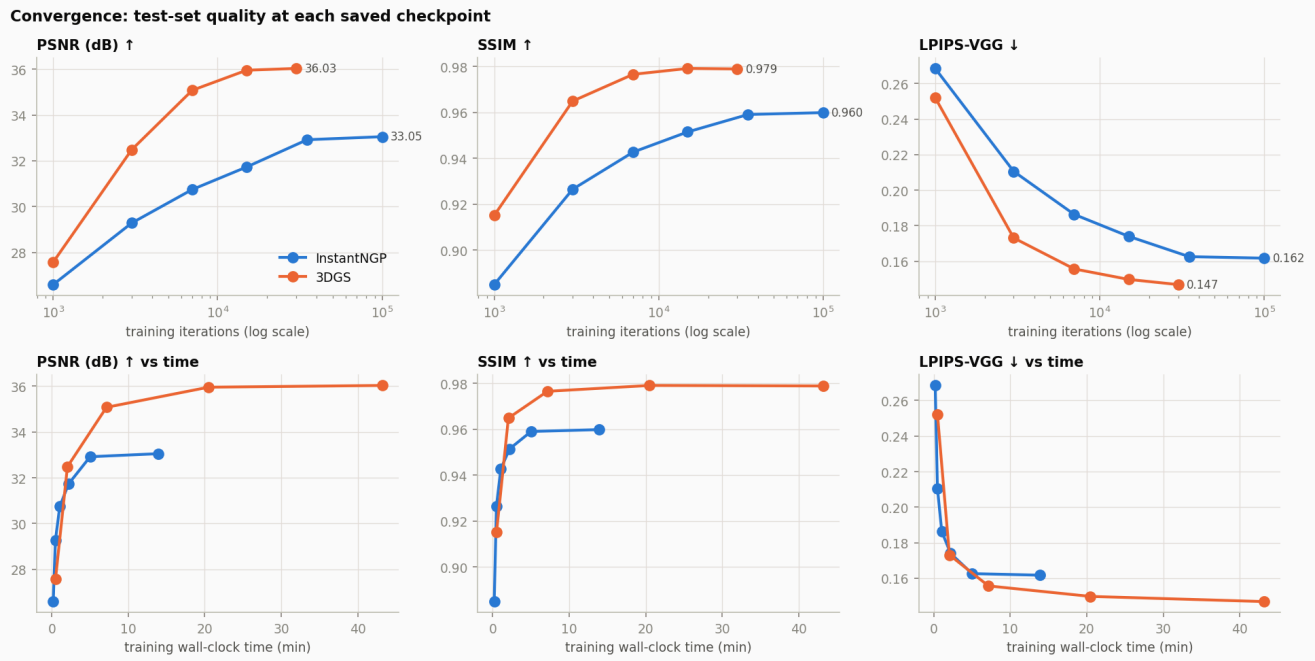


Figure 5. Test metrics against iterations (top, log scale) and against wall-clock training time (bottom).

Observations:

- **Per unit of time, InstantNGP is ahead only in the first minute.** At about 0.5 min it reaches 29.3 dB, against 27.6 dB for 3DGS. By 2 min, 3DGS is ahead (32.5 vs about 31.7 dB). After 7 min, 3DGS (35.1 dB) exceeds the best InstantNGP result at any training length (33.05 dB).
- **Half of 3DGS's training time adds 0.08 dB.** The Gaussian count stops changing at 15k iterations, when densification ends. The last 15k iterations take 22.7 of the 43.2 minutes.
- **3DGS fits its training views much better than the test views.** Its own logs at 30k report 40.30 dB on training views and 36.06 dB on test views (a 4.2 dB gap). The 36.06 dB is computed on float renders; our 36.03 dB is computed on the saved 8-bit PNGs.

5.4 Training time

	InstantNGP 35k	InstantNGP 100k	3DGS 30k	3DGS 15k
Wall clock	301 s	834 s	2594 s	1229 s
Relative to InstantNGP 35k	1.0x	2.8x	8.6x	4.1x

InstantNGP trains 8.6x faster at its default schedule. 3DGS's cost per iteration grows with the number of Gaussians: the 1k checkpoint is written 29 s after start (including data loading), while iterations 15k–30k take (2591 − 1229) / 15000 = 91 ms each.

5.5 Rendering time

	ms / frame (mean ± std)	FPS
3DGS	17.4 ± 1.7	57.6
InstantNGP, 1 spp	20.3 ± 1.6	49.2
InstantNGP, 8 spp (setting used for the quality metrics)	155.8 ± 12.5	6.4

At the settings that produce the reported quality, 3DGS renders 9.0x faster. At 1 spp the two methods are close in speed on this laptop GPU, but InstantNGP's 1 spp images contain more sampling noise than the 8 spp images that were evaluated. 3DGS rasterises 1.15 M Gaussians. InstantNGP marches rays through its occupancy grid and evaluates the hash encoding and two MLPs per sample.

5.6 Model size

	InstantNGP	3DGS
File on disk	23.4 MB (.ingp, compressed)	271.8 MB (.ply)
Uncompressed	36.6 MB	271.8 MB
Content	12.87 M parameters (fp16): hash tables 12,855,296, MLPs 10,240; plus fp16 occupancy grid (3 x 128^3 cells)	1,148,987 Gaussians x 62 float32 values (position, normal, 48 SH colour coefficients, opacity, 3 scales, 4 rotation)

The uncompressed InstantNGP size matches its contents: 12,865,536 x 2 bytes + 3 x 128^3 x 2 bytes = 38.31 x 10^6 bytes, against 38.35 x 10^6 bytes measured (the rest is configuration metadata).

3DGS is 11.6x larger on disk. 48 of its 59 learned values per Gaussian are spherical-harmonic colour coefficients. The InstantNGP model has a fixed size from the first step, while 3DGS grows during densification (Figure 6).

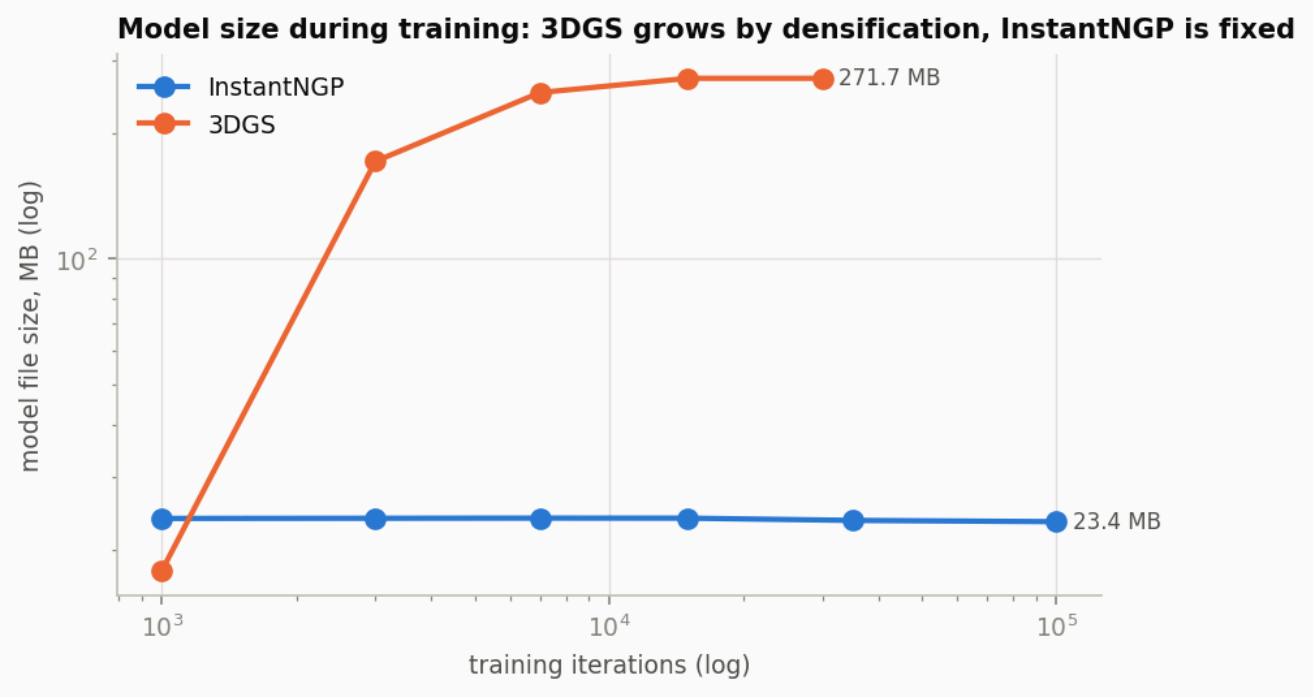


Figure 6. Model file size during training.

5.7 Visual comparison

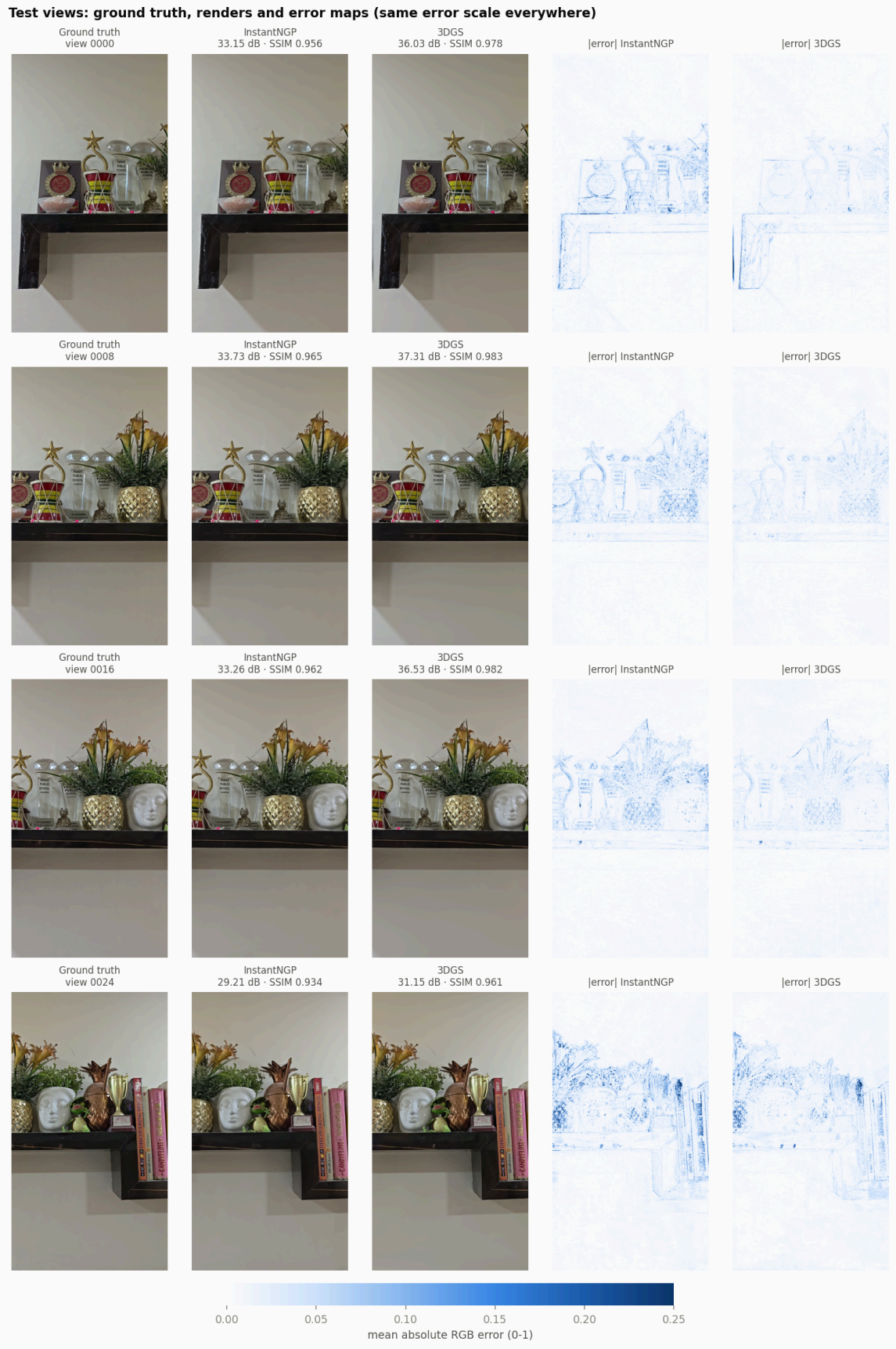


Figure 7a. Test views 0000–0024: ground truth, InstantNGP, 3DGS and absolute error maps (common scale).

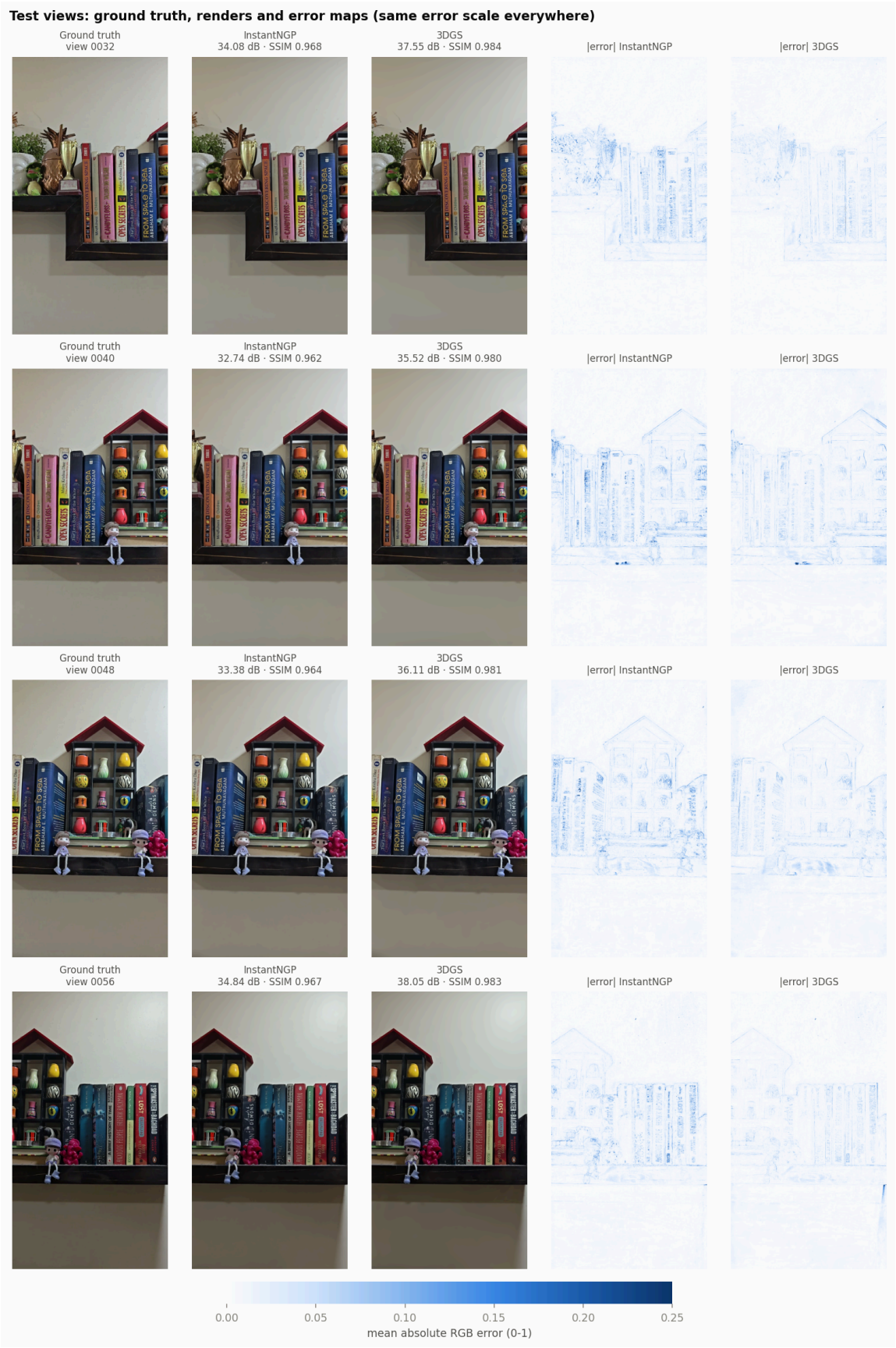


Figure 7b. Test views 0032–0056.

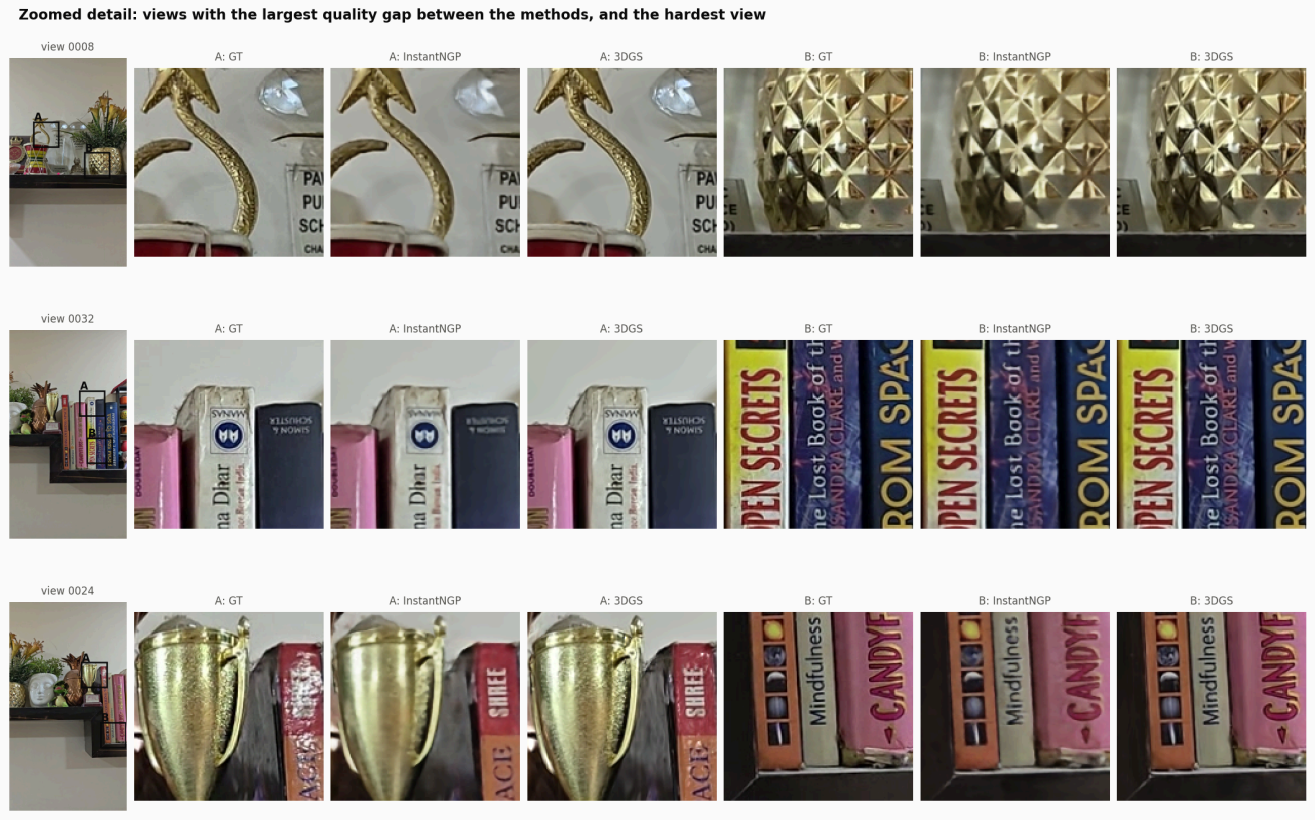


Figure 8. Crops at the highest-detail regions of the views with the largest PSNR gap (0008, 0032) and of the hardest view (0024).

Observations:

- Both methods reproduce the scene geometry, colours and the static shadows under the shelf. Neither shows floaters in the test views.
- **Fine texture and text:** 3DGS reproduces small book-spine text ("Simon & Schuster", barcodes) and the facets of the gold pot close to the ground truth. In InstantNGP the same details are soft and small text is not readable (Figure 8, rows 1–2).
- **Shiny surfaces:** on the trophy (view 0024) the ground truth shows a fine specular texture. 3DGS keeps most of it; InstantNGP renders a smooth, blurred highlight.
- **Error maps:** both concentrate on edges and textured objects. The plain wall has near-zero error for both.

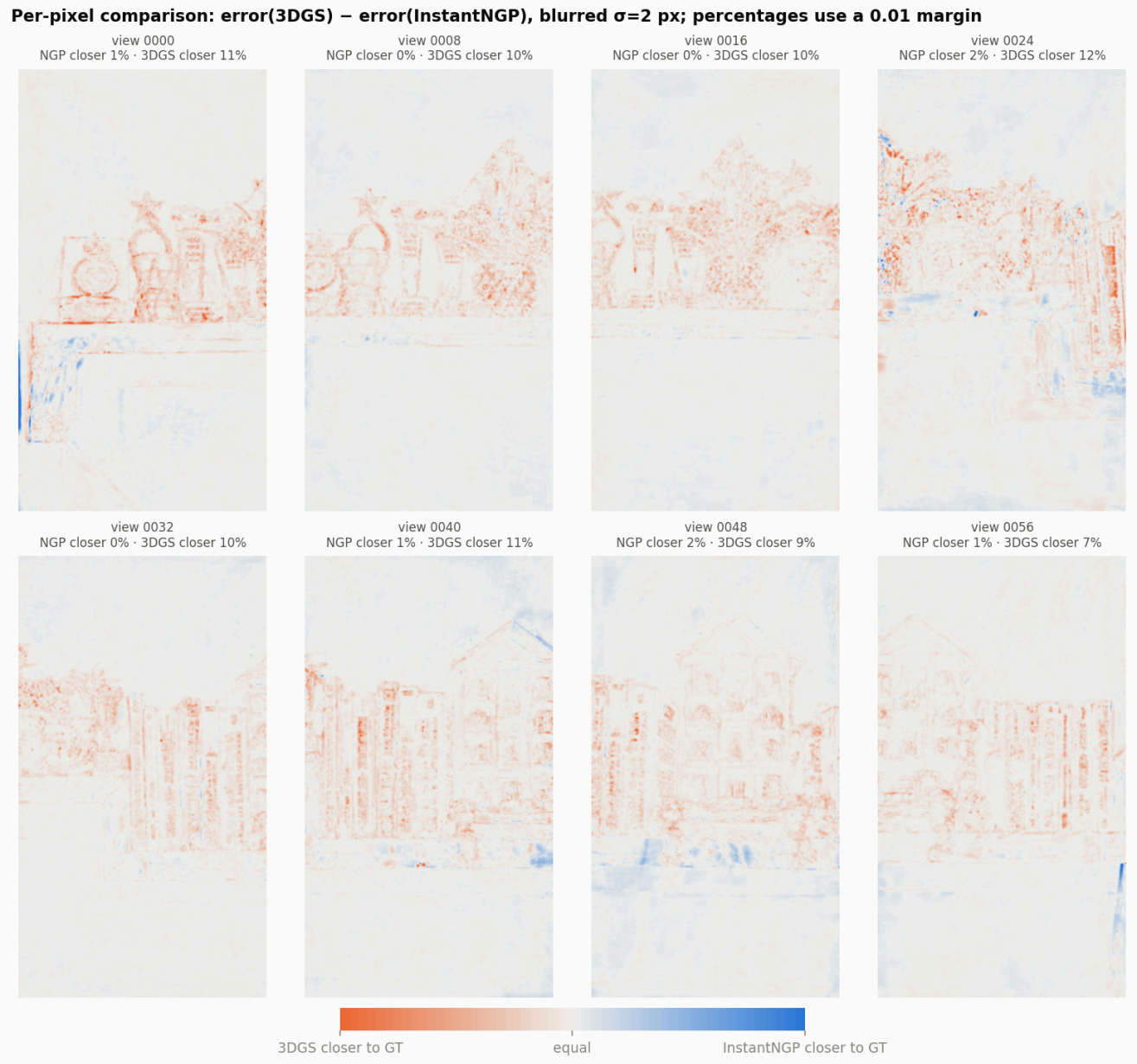


Figure 9. Per-pixel error(3DGS) − error(InstantNGP), blurred with σ = 2 px. Orange: 3DGS closer to ground truth; blue: InstantNGP closer.

With a 0.01 margin, 3DGS is closer to the ground truth on 7–12 % of pixels per view and InstantNGP on 0–2 %. The remaining pixels (mostly the wall) are equal. The regions where InstantNGP is closer are small and mostly at image borders and the shelf's lower edge (views 0000, 0048, 0056).

5.8 Error and frequency analysis

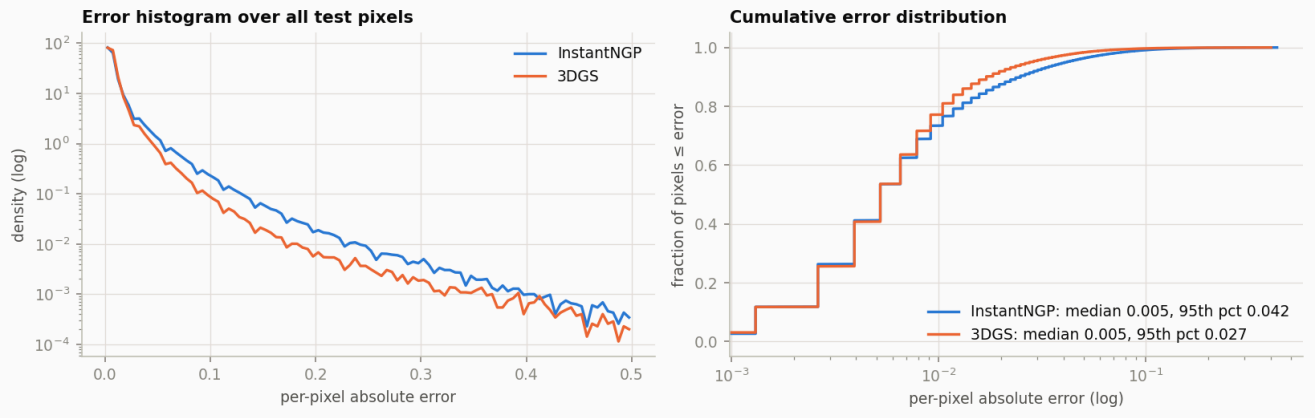


Figure 10. Distribution of per-pixel absolute error over all test pixels.

The median per-pixel error is the same (0.005) for both methods. The difference is in the tail: the 95th percentile is 0.027 for 3DGS and 0.042 for InstantNGP. Large errors are rarer in 3DGS at every error level above about 0.02.

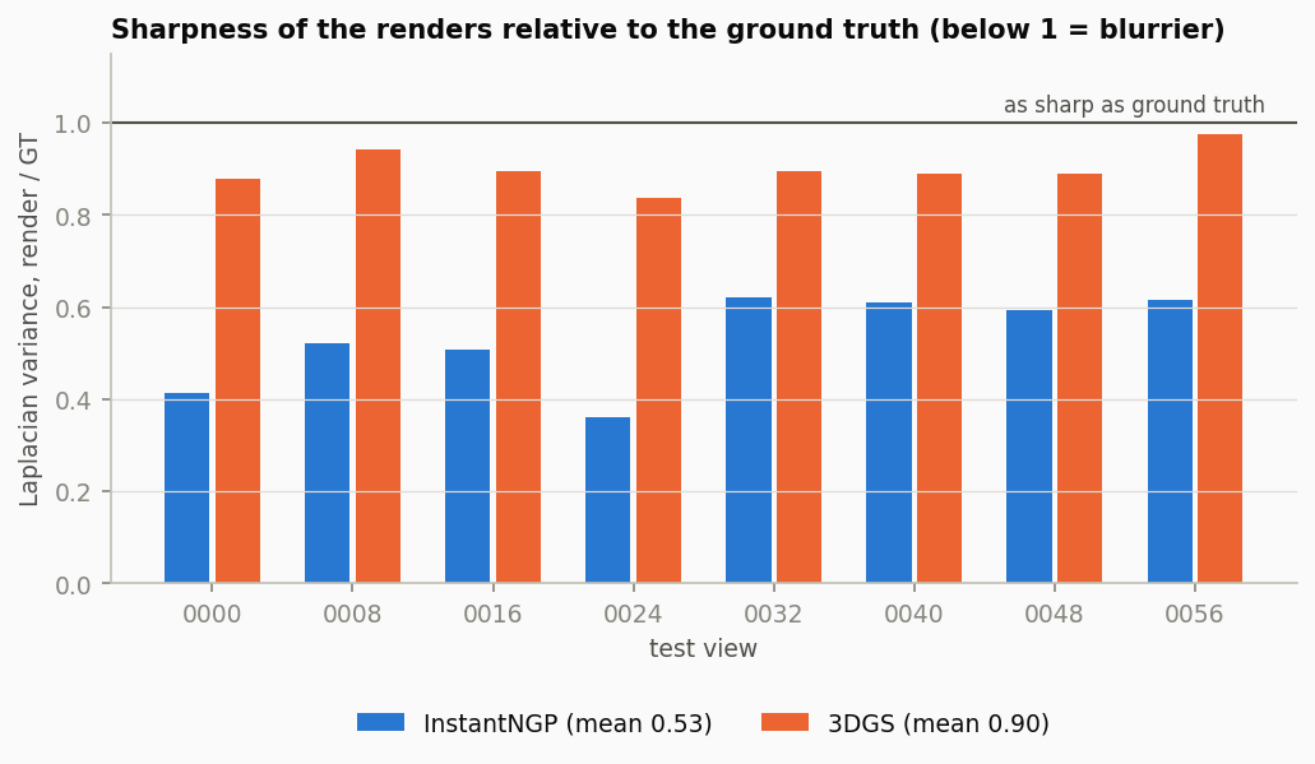


Figure 11. Variance of the Laplacian of each render relative to the ground truth.

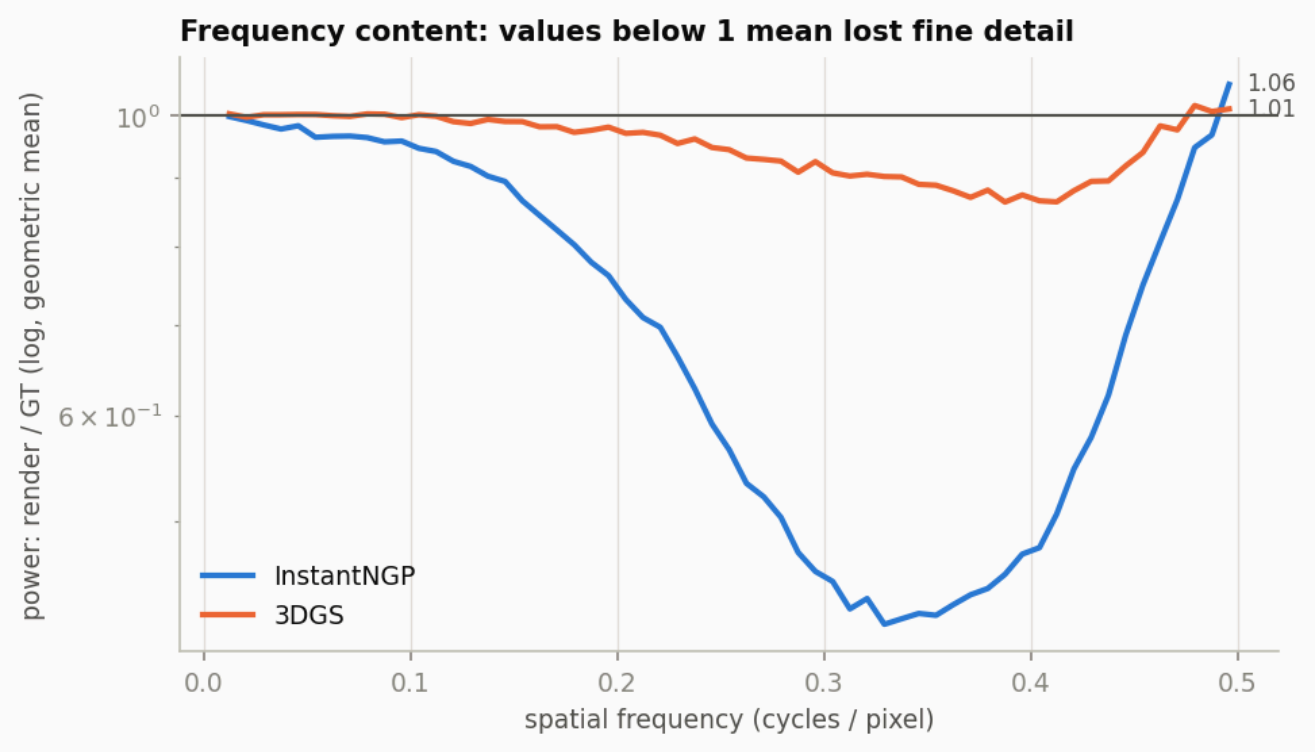


Figure 12. Radial power spectrum of the renders divided by that of the ground truth (geometric mean over test views).

- **Sharpness:** InstantNGP renders keep 53 % of the ground-truth Laplacian variance on average, 3DGS keeps 90 %.
- **Frequency content:** 3DGS keeps at least about 85 % of the ground-truth power at all frequencies. InstantNGP drops to about 45 % between 0.3 and 0.4 cycles/pixel. This is the loss of fine detail seen in Figure 8.
- **Near the Nyquist limit** (> 0.45 cycles/pixel), InstantNGP's power rises back above the ground truth (1.06). That is pixel-level noise from stochastic ray sampling, not recovered detail.

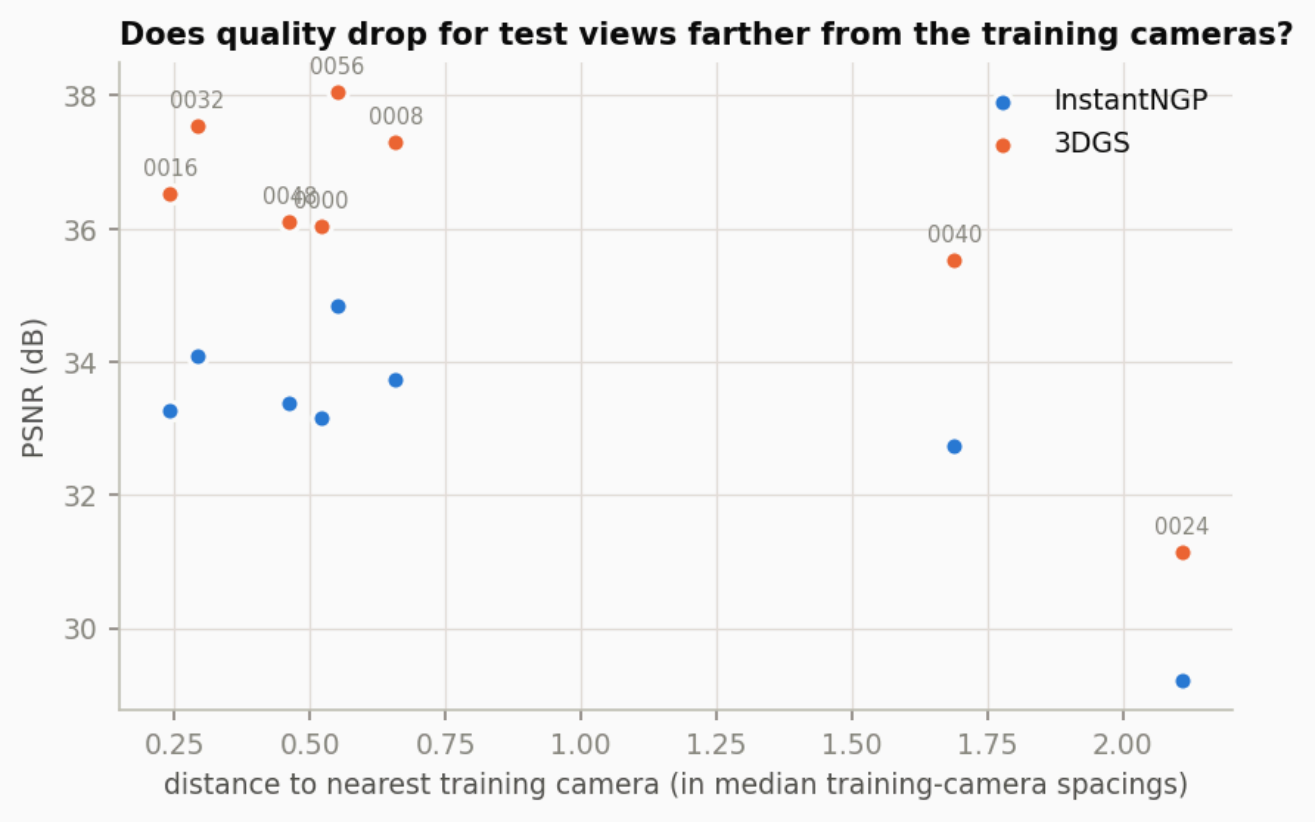


Figure 13. Test PSNR against distance to the nearest training camera (in units of the median spacing between training cameras).

- Six test views lie within 0.7 camera spacings of a training camera.
- Views 0040 (1.7 spacings) and 0024 (2.1 spacings) are farther, because the capture speed was uneven there. View 0024 is the worst view for both methods (31.15 / 29.21 dB), and view 0040 is below the mean for both.
- **Both methods lose quality with the distance to the nearest training view.** The 3DGS advantage holds at every distance.

5.9 Novel views away from the capture path

Seven camera paths were rendered with identical cameras for both methods (Figure 14):

Path	Description
trajectory	the captured path, smoothed (interpolation)
shift_up / shift_down	captured path moved up / down by 0.15 x scene depth
spiral	captured path plus a circular offset of 0.08 x scene depth, 3 turns
dolly	mid-path camera moving ±0.35 x scene depth along the viewing axis
orbit	±30° arc around the scene centre, looking at it
look_around	mid-path camera panning ±20° and tilting ±12°

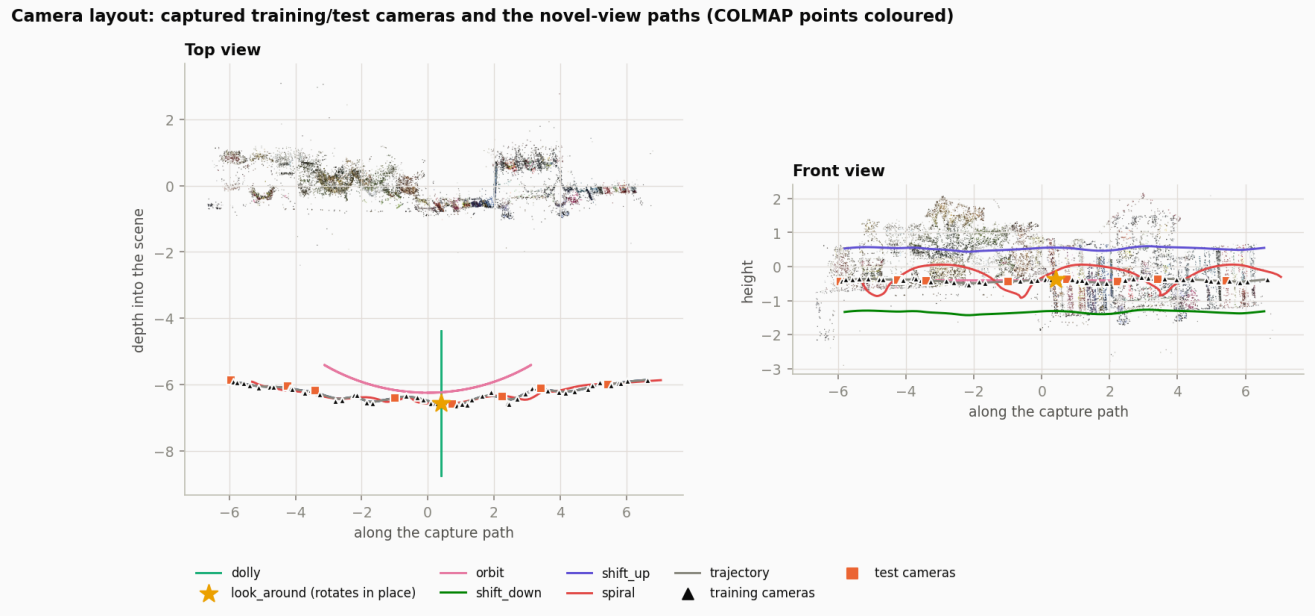


Figure 14. Training cameras (triangles), test cameras (squares) and novel paths, over the COLMAP points.

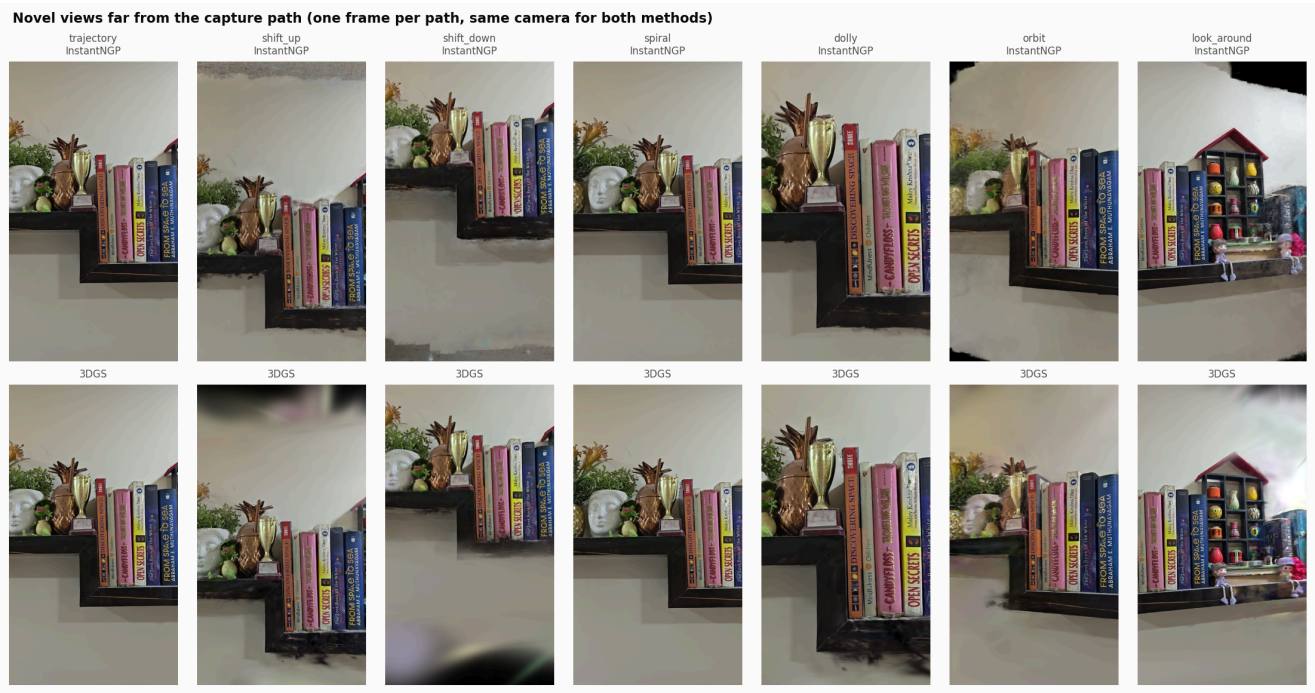


Figure 15. One frame per path at its largest deviation from the captured views; top InstantNGP, bottom 3DGS, same camera.

Per-path frame strips: `results/figures/13_novel_<path>.png`.

- **trajectory, spiral and dolly** (inside or close to the captured views): both methods give clean images; 3DGS is sharper, matching the test-view results.
- **shift_up / shift_down** expose wall and shelf regions never seen by the cameras.
- InstantNGP fills them with grainy, noisy texture.
- 3DGS shows smooth dark or coloured blobs at the frame borders (large, low-opacity Gaussians that were never constrained from these directions).
- **orbit and look_around** point the camera partly outside the captured volume.
- InstantNGP renders black where rays leave its bounding box (`aabb_scale` 4).
- 3DGS shows stretched, smeared splats on the side of the shelf.
- The shelf itself stays recognisable in both.

- A single horizontal pass gives almost no vertical parallax, so neither method can recover surfaces that are hidden from the capture line (tops of books, the area behind objects).

5.10 3DGS model statistics

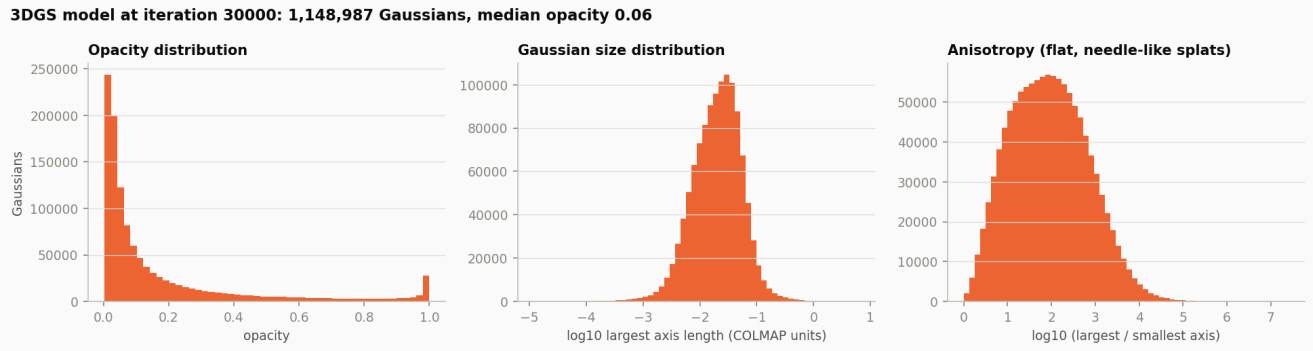


Figure 16. Opacity, size and anisotropy of the 1,148,987 Gaussians at 30k iterations.

- **Count:** densification grew the 16,573 SfM points to 1,148,987 Gaussians (69x).
- **Opacity:** median 0.06. Most Gaussians are nearly transparent and contribute through accumulation; a separate group is fully opaque (peak at 1.0).
- **Size:** the largest axis is typically 10^-1.6 COLMAP units, about 0.4 % of the scene depth.
- **Shape:** the ratio of largest to smallest axis has its mode around 100:1. The scene is represented mostly by flat, disc- or needle-like splats aligned with surfaces.

6. Discussion

Criterion	Better method	Margin on this scene
Test-view quality	3DGS	+2.98 dB PSNR, +0.019 SSIM, −0.015 LPIPS-VGG, −0.022 LPIPS-Alex; better on all 8 views
Rendering time at evaluated quality	3DGS	17.4 ms vs 155.8 ms (9.0x)
Model size	InstantNGP	23.4 MB vs 271.8 MB (11.6x smaller)
Training time to convergence	InstantNGP	5.0 min vs 43.2 min (8.6x); 3DGS is 0.08 dB from its final quality after 20.5 min
Training GPU memory	InstantNGP	1.2 GB vs 4.3 GB (whole-GPU peak)

- **Representation.** 3DGS stores an explicit, adaptive set of anisotropic primitives. Their number grew to 1.15 M exactly where the scene has detail (text, leaves, faceted objects), which gives sharp renders and high-frequency fidelity but a large model. InstantNGP stores a fixed hash grid (12.9 M parameters, shared through hashing) and two small MLPs. It is compact and fast to train, but its renders lose fine detail (Figures 11, 12).
- **Rendering.** Rasterising sorted Gaussians costs one pass per image. Volume rendering costs many network evaluations per ray, and InstantNGP needs several samples per pixel to reach the quality reported here.
- **Shiny surfaces.** On the trophy and the gold pot, InstantNGP produces smooth highlights. 3DGS keeps more of the fine specular texture, but its view-dependent colour (degree-3 spherical harmonics) can only interpolate between the captured directions.

- **Extrapolation.** Both methods are reliable only near the captured line. Their failure modes differ: noise and empty space for InstantNGP, smeared splats for 3DGS.

7. Limitations

1. **Shadows and reflections.** The scene contains static soft shadows below the shelf and specular highlights on the ceramic, glass and metal objects. They are static and consistent across views, but the shiny objects are where both methods have their largest local errors.
2. **Small test set.** 8 test views from a single sideways pass, all close to the training cameras. The metrics measure interpolation quality, not wide-baseline novel-view synthesis.
3. **Laptop GPU under WSL2.** All numbers come from one RTX 4060 Laptop GPU under WSL2. GPU memory is the whole-GPU peak, because WSL2 does not report per-process usage. Absolute times would be lower on a desktop GPU; the ratios between the methods are the comparable quantity.
4. **InstantNGP render speed depends on spp.** 1 spp is 7.7x faster than 8 spp but noisier; quality was measured only at 8 spp.
5. **One scene, one run per method.** No variance over random seeds was measured.

8. Reproducibility

Item	Location
Frames, COLMAP model, splits, novel paths	`data/scene/`, `results/train_list.txt`, `results/test_list.txt`
Pipeline code	`scripts/` (`filter_frames.py`, `prepare_ngp.py`, `make_novel_paths.py`, `train_ngp.py`, `render_gs.py`, `render_ngp.py`, `eval_metrics.py`, `collect_results.py`, `make_figures.py`)
Full run	`bash scripts/run_all.sh` (settings in `scripts/config.sh`)
Raw results	`results/metrics/*.json`, `results/timing/*.json`, `results/summary.json`, `logs/`
Model repositories	gaussian-splatting `54c035f7834b564019656c3e3fcc3646292f727d`, instant-ngp `abe236ee00cf90cfca6e36e65c00435d5b21f50a`
COLMAP	4.2.0; `convert.py` option names updated for COLMAP 4 (`scripts/patches/gs_convert_colmap4.patch`)

Main commands:

```
python repos/gaussian-splatting/convert.py -s data/scene --no_gpu        # COLMAP (run once)
python scripts/prepare_ngp.py                                            # InstantNGP poses + split
python repos/gaussian-splatting/train.py -s data/scene -m outputs/gs --eval -r 1 \
       --iterations 30000 --save_iterations 1000 3000 7000 15000 30000 --disable_viewer
python scripts/train_ngp.py --scene data/scene/transforms_train.json --out_dir outputs/ngp \
       --checkpoints 1000 3000 7000 15000 35000 100000
python scripts/eval_metrics.py --pred_dir outputs/eval/<method>/<checkpoint> --out
results/metrics/<name>.json
```

References

[1] T. Müller, A. Evans, C. Schied, A. Keller. Instant Neural Graphics Primitives with a Multiresolution Hash Encoding. ACM Transactions on Graphics (SIGGRAPH), 2022.

[2] B. Kerbl, G. Kopanas, T. Leimkühler, G. Drettakis. 3D Gaussian Splatting for Real-Time Radiance Field Rendering. ACM Transactions on Graphics (SIGGRAPH), 2023.

[3] J. L. Schönberger, J.-M. Frahm. Structure-from-Motion Revisited. CVPR, 2016.

[4] R. Zhang, P. Isola, A. A. Efros, E. Shechtman, O. Wang. The Unreasonable Effectiveness of Deep Features as a Perceptual Metric (LPIPS). CVPR, 2018.